TRABAJO PRÁCTICO — AVANCE TP4 (SP4 y SP5, primera parte)

Integración con JavaScript

Caso de estudio: Nutrite — Dietética Online

Materia	Aplicaciones Web I — Sistemas
Profesor	Andrés Senn
Alumno	Giuliano Baldini
Fecha	Noviembre de 2026 — Córdoba
Repositorio	github.com/Jbaigorria22/aw1-tp (rama `giuliano-vistas`)

1. Introducción y alcance

En el SP3 Nutrite quedó maquetado en HTML5 y estilizado con CSS sin JavaScript. En esta primera parte del TP4 se incorpora JavaScript (módulos ES, `fetch`, `localStorage` y eventos del DOM) para que el sitio deje de ser estático, y se completan todas las vistas que la propuesta planteaba, en particular el proceso de compra por pasos.

Requisito de la consigna	Qué se hizo	Estado
Renderizado del catálogo vía JavaScript	La grilla se dibuja desde `render.js` con los datos de `productos.json`.	Hecho
Datos del catálogo en un módulo independiente	`datos.js` + `assets/data/productos.json`.	Hecho
Organizar y modularizar el JavaScript	Un módulo por responsabilidad (sección 3).	Hecho
Eventos JavaScript	Filtros, buscador, orden, carrito, pasos del checkout, validaciones (sección 4).	Hecho
Terminar las vistas planteadas	Checkout de 4 pasos, ficha de producto dinámica, inicio dinámico y contacto con validación (sección 2).	Hecho
Informe PDF con cambios respecto de la propuesta	Este documento (sección 5).	Hecho

Cómo ver el proyecto

Como se usan módulos ES y `fetch`, el sitio debe abrirse con un servidor local (no con doble clic sobre el HTML). Por ejemplo, desde la carpeta del proyecto: `python -m http.server 8000` y abrir `http://localhost:8000/index.html`, o la extensión Live Server de VS Code.

2. Vistas desarrolladas

Vista	Archivo	Novedad en el TP4
Inicio	`index.html`	Los productos destacados se renderizan con JS (`inicio.js`) y el contador del carrito refleja el estado real.
Catálogo	`catalogo.html`	Renderizado por JS, filtros, buscador, orden, "Agregar al carrito" y búsqueda desde el header (`?q=`).
Ficha de producto	`producto.html`	Dinámica: lee `?id=` y muestra el producto elegido (antes era un único producto fijo).
Carrito	`carrito.html`	Cantidades, eliminación, envío y totales con JS (trabajo de equipo, rama `matias-carrito-checkout`).
Checkout	`checkout.html`	**Los 4 pasos del stepper**: Datos, Entrega, Pago y Confirmación (antes solo existía el paso Pago).
Contacto	`contacto.html`	Validación del formulario con mensajes de error y confirmación.
Blog	`blog.html` , `blog-articulo.html`	Sección nueva: listado de artículos filtrable y vista de cada artículo con productos relacionados (sección 2.4).

2.1 Checkout por pasos

El proceso de compra planteado en la propuesta (stepper con 4 etapas) quedó completo en una sola página: `checkout.html` contiene un panel por paso y `checkout.js` muestra uno solo por vez, actualizando el stepper (paso activo, pasos completados y `aria-current`).

Paso	Contenido	Validaciones
1. Datos	Nombre y apellido, email, teléfono, DNI.	Nombre ≥ 3 caracteres; email con formato válido; teléfono de 8 a 12 números; DNI de 7 u 8 números.
2. Entrega	Moto, correo o retiro en punto Nutrite; dirección (calle, ciudad, CP, aclaraciones).	Si el envío es "Retiro" se oculta la dirección; si no, calle, ciudad y CP de 4 números son obligatorios.
3. Pago	Crédito (con tabla de cuotas), débito, transferencia (10% OFF) y billetera.	Total y cuotas se recalculan al cambiar de medio o de plan.
4. Confirmación	Número de pedido (`NUT-######`), datos de entrega, medio de pago y total pagado. Vacía el carrito.	—

El resumen del pedido (columna lateral) se actualiza en tiempo real con el tipo de envío y el medio de pago elegidos. Cada paso permite volver al anterior sin perder lo cargado. Los datos del cliente se escapan antes de insertarse en el HTML de la confirmación para evitar inyección de código.

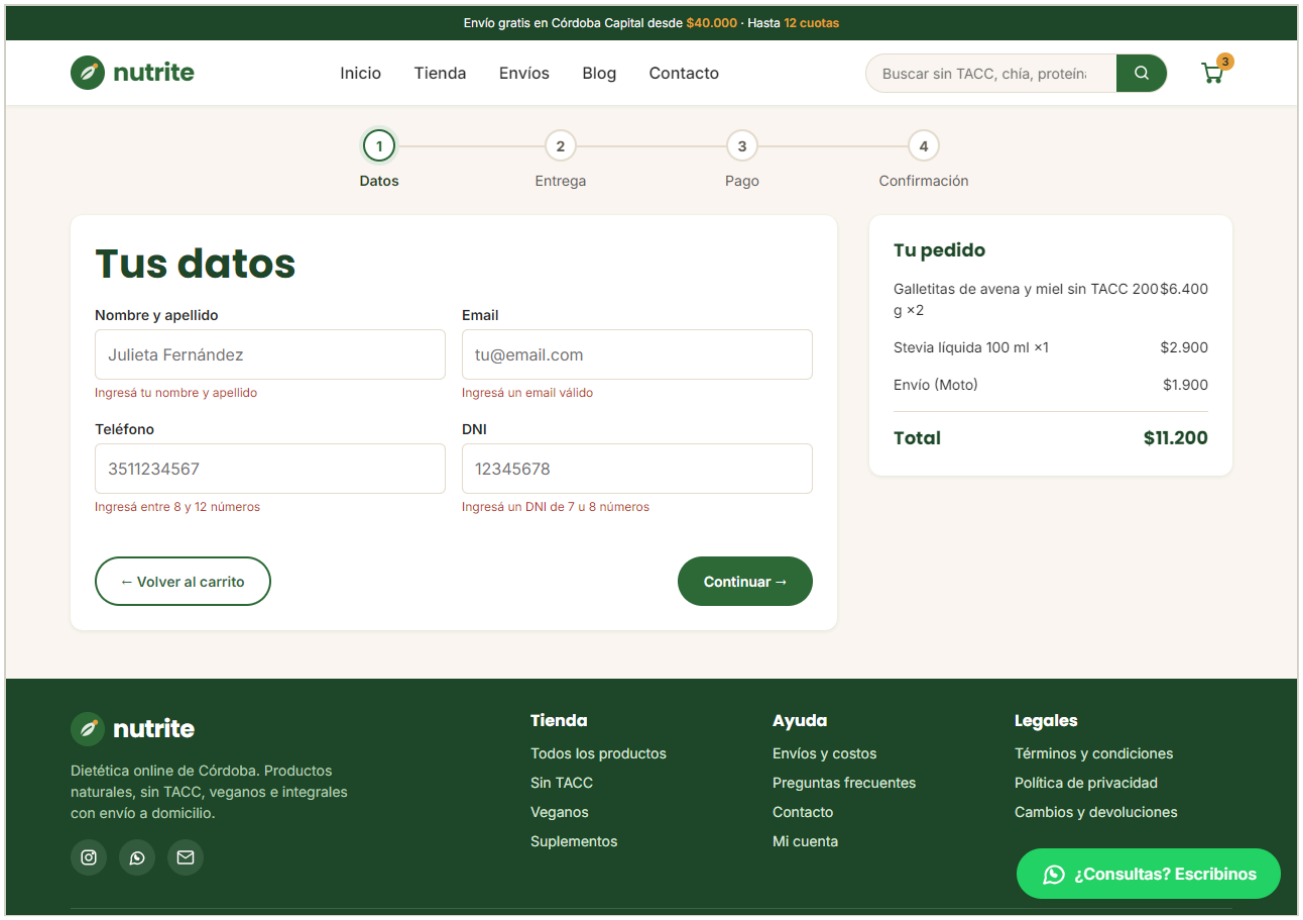


Figura 1 — Paso 1 (Datos) con validación de campos.

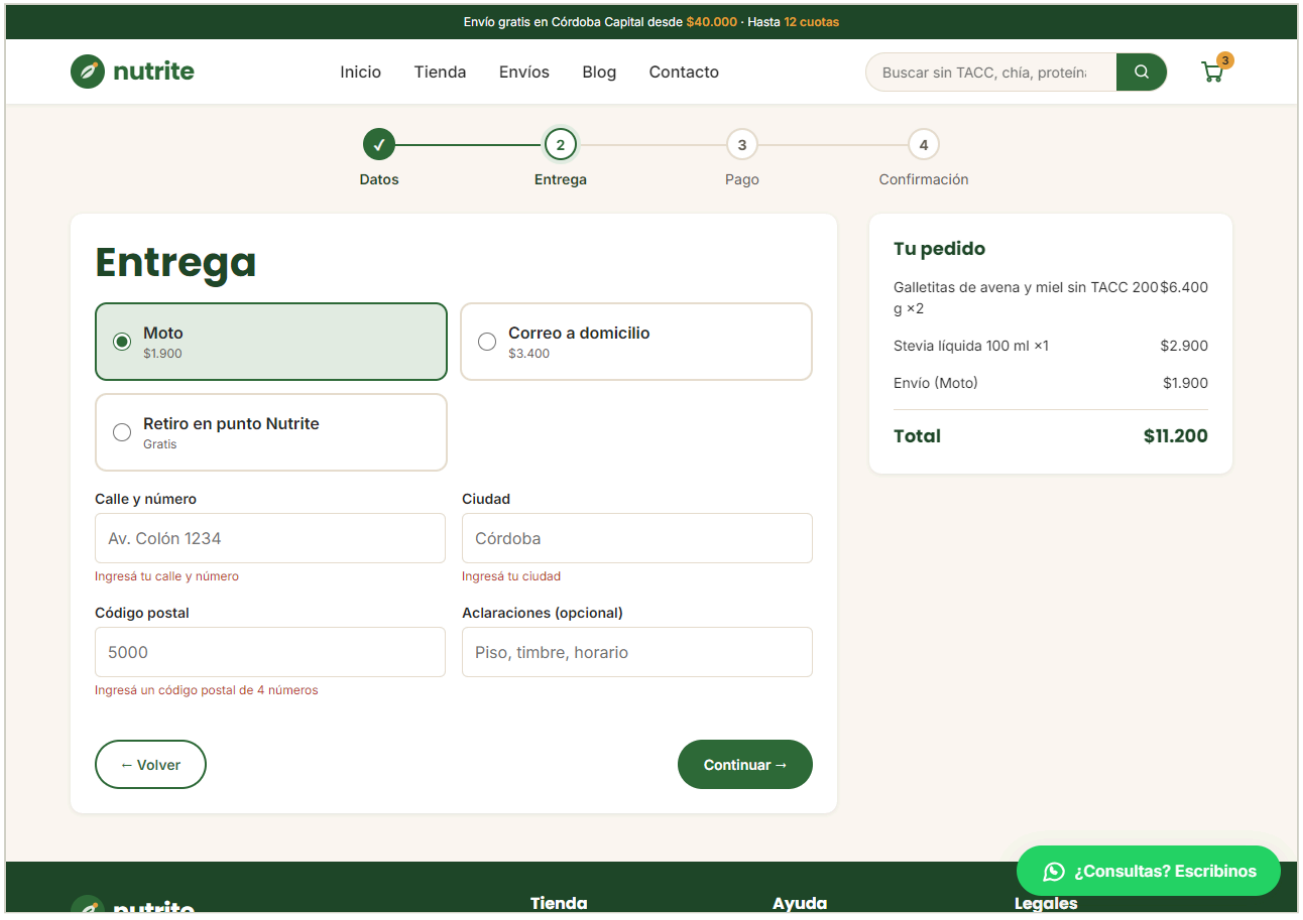


Figura 2 — Paso 2 (Entrega): opciones de envío, dirección y resumen lateral.

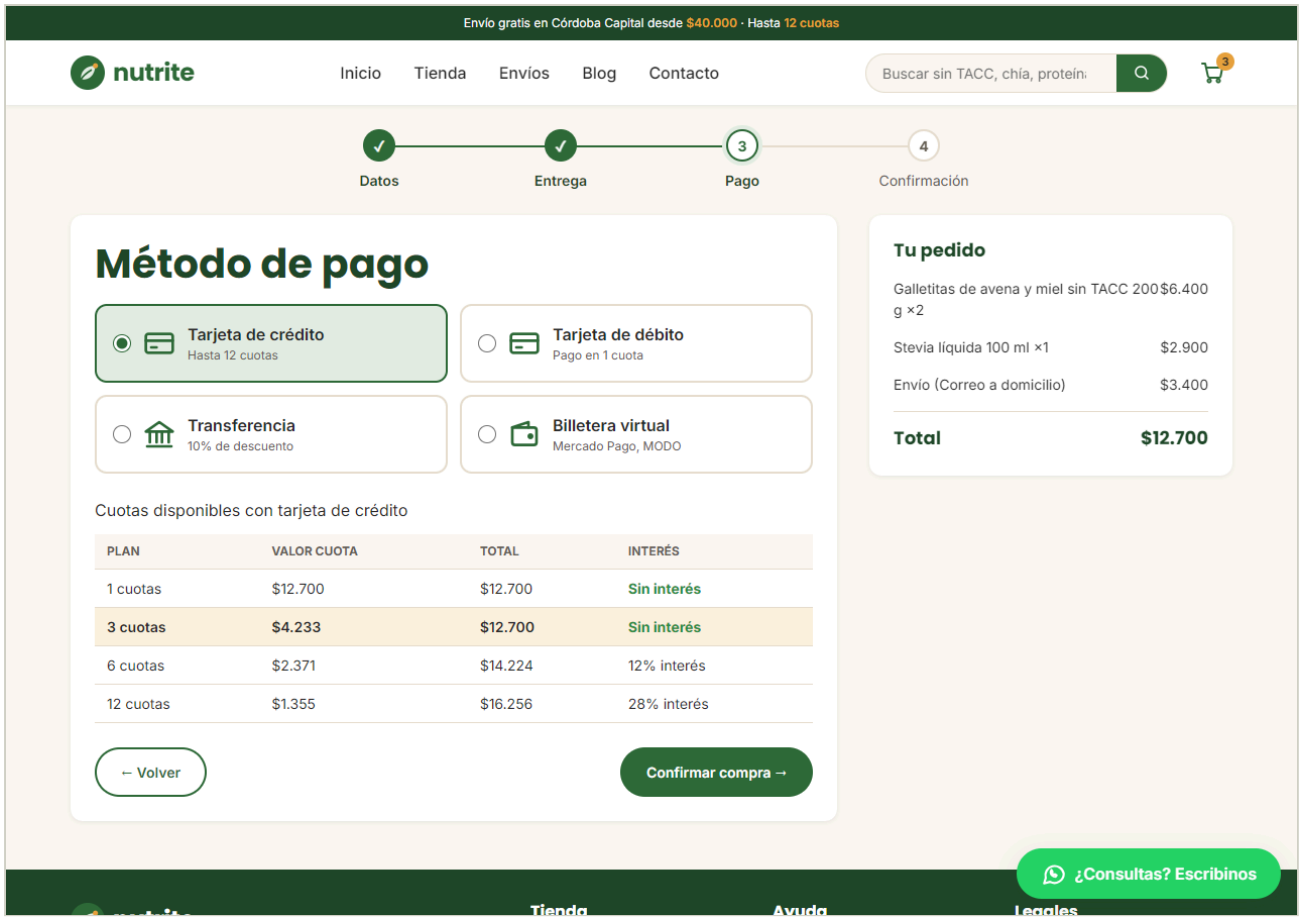


Figura 3 — Paso 3 (Pago): medios de pago y cuotas.

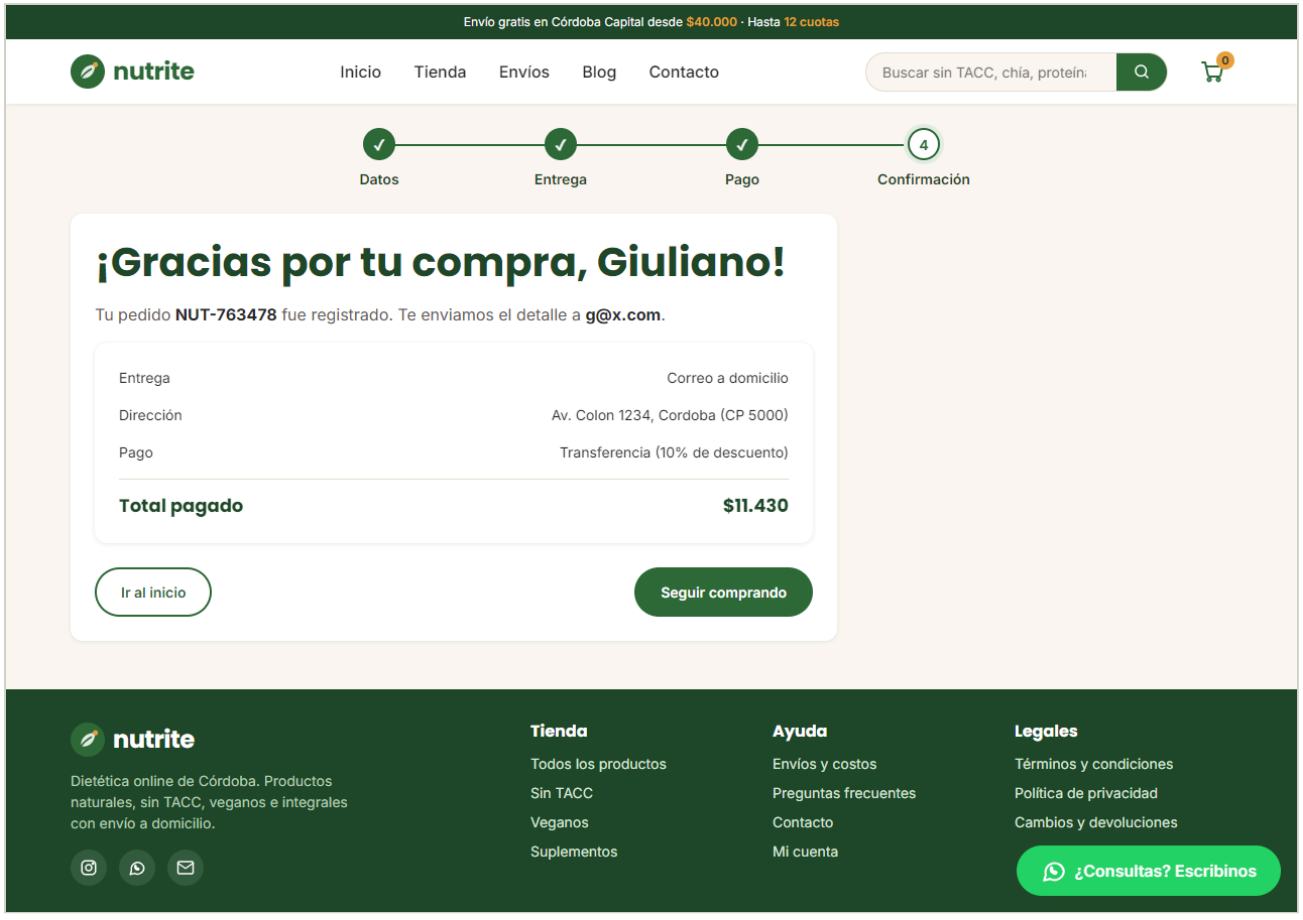


Figura 4 — Paso 4 (Confirmación) con número de pedido.

2.2 Ficha de producto dinámica

`producto.js` lee el parámetro `id` de la URL (`producto.html?id=2`), busca el producto en el módulo de datos y dibuja la ficha: galería con miniaturas, badges, stock, precio, cuotas, precio con transferencia, selector de cantidad (limitado al stock), cálculo de envío por código postal, botón de WhatsApp con el nombre del producto en el mensaje y acordeones de descripción e ingredientes. Los productos relacionados (misma categoría primero) se renderizan con la misma tarjeta del catálogo. Si el `id` no existe se muestra un mensaje; si el producto no tiene stock, el botón de compra queda deshabilitado.

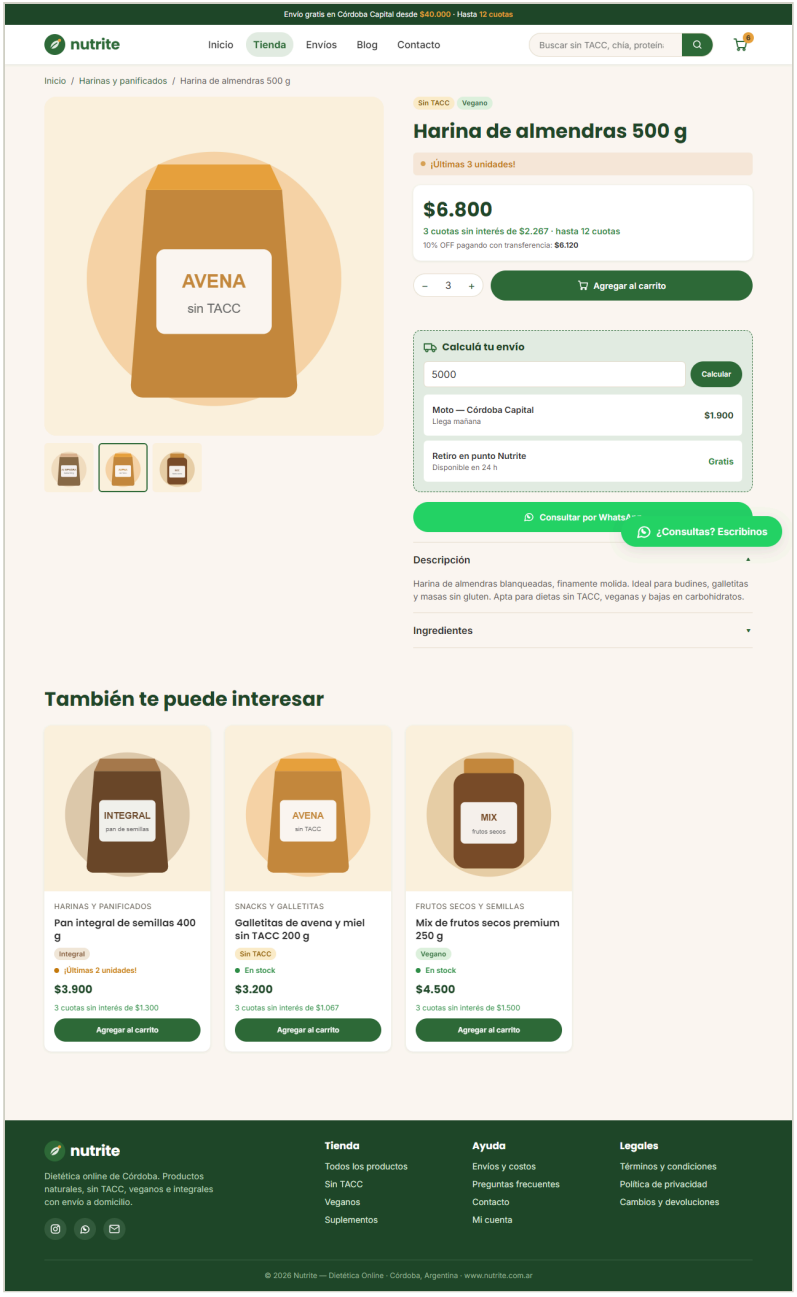


Figura 5 — Ficha de producto generada por JavaScript.

2.3 Inicio, catálogo y contacto

- **Inicio:** los 4 primeros productos con stock se dibujan desde el mismo JSON, reutilizando `crearTarjetaProducto` .
- **Catálogo:** las tarjetas enlazan a `producto.html?id=N` y el botón "Agregar al carrito" suma al carrito y muestra "¡Agregado!". El buscador del header de cualquier página llega a `catalogo.html?`

`q=texto` y filtra al abrir.

- **Contacto:** validación de nombre, email y mensaje (≥ 10 caracteres) con marcado de errores (`campo--error`) y mensaje de confirmación.

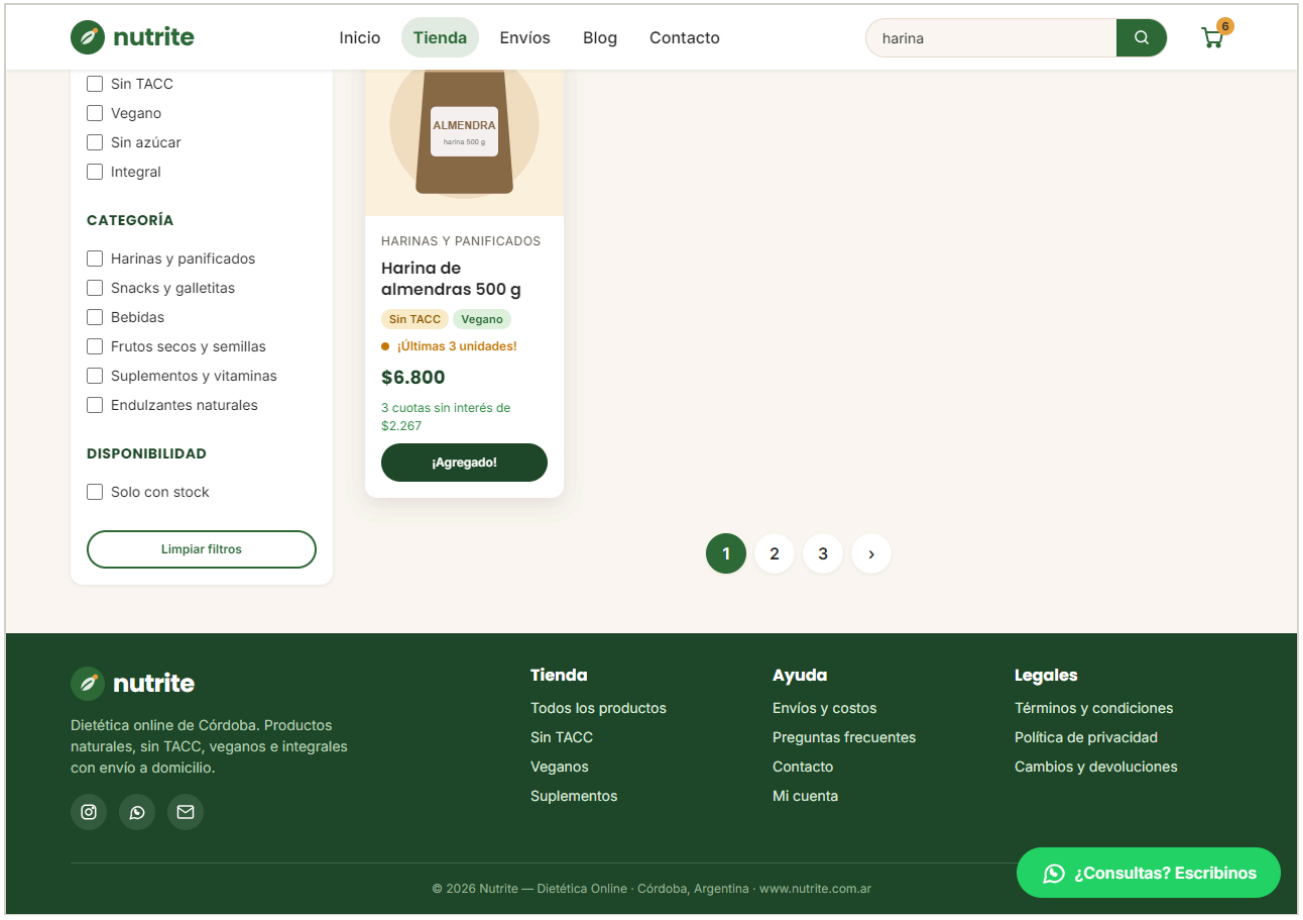


Figura 6 — Catálogo renderizado por JavaScript (búsqueda `?q=harina`).

2.4 Blog

Se agregó una sección de blog simple, enlazada desde el menú "Blog" de todas las páginas (antes el enlace era un marcador `#`). Contiene 6 artículos de ejemplo (recetas, tips y novedades) guardados en `assets/data/articulos.json`, con título, categoría, fecha, resumen, texto y los ids de los productos relacionados.

- **Listado** (`blog.html` + `blog.js`): tarjetas dibujadas por JS, chips de categoría (Todos / Recetas / Tips / Novedades), buscador por título con filtrado al escribir y contador de resultados.
- **Artículo** (`blog-articulo.html?id=N` + `articulo.js`): lee el id de la URL, muestra el texto y, al final, tarjetas de los productos del catálogo mencionados, con "Agregar al carrito" (reutiliza `render.js` y `agregar.js`). Si el id no existe se muestra un mensaje.
- **Módulos:** `datos.js` suma `obtenerArticulos()` y `render-blog.js` contiene el dibujo de tarjetas y artículo, siguiendo la misma separación datos / render / punto de entrada del catálogo.
- **Eventos:** `click` en los chips de categoría, `input` en el buscador y `submit` del buscador (se evita la recarga).

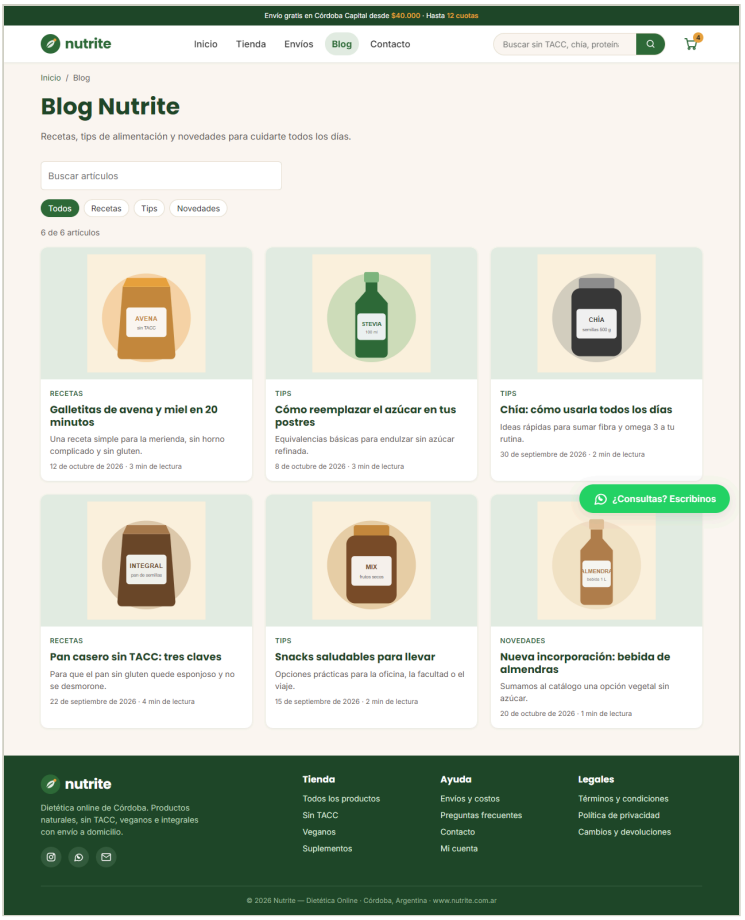


Figura 7 — Listado del blog con filtros y buscador.

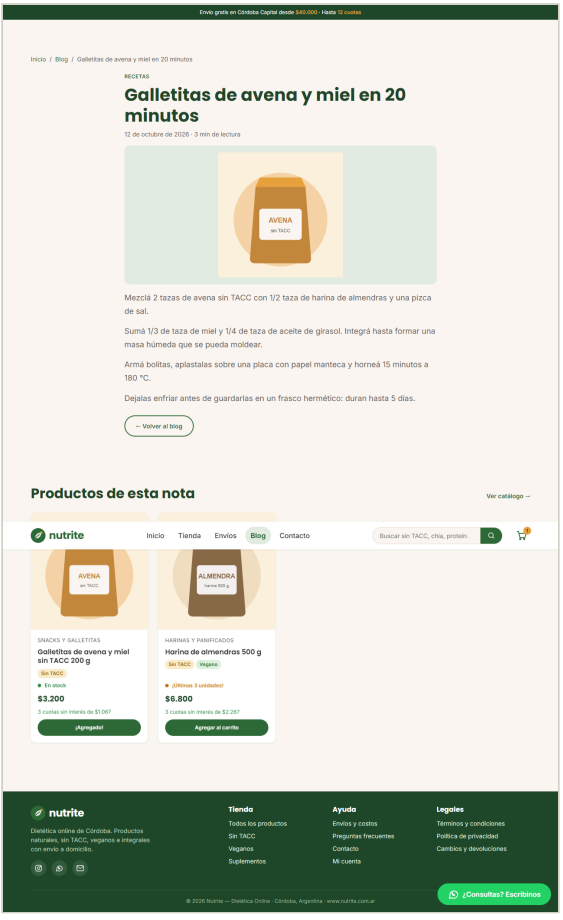


Figura 8 — Artículo con productos relacionados.

3. Organización y modularización del JavaScript

Todo el código vive en `assets/js/` y usa módulos ES (`<script type="module">`). Cada archivo tiene una sola responsabilidad; las páginas tienen un archivo "punto de entrada" que coordina los módulos.

Módulo	Responsabilidad	Usado por
`datos.js`	Único lugar que sabe de dónde salen los productos (`fetch` a `productos.json`).	Todas las páginas con productos
`render.js`	Convierte un producto en una tarjeta HTML; formatea precios.	catálogo, inicio, producto
`filtros.js`	Funciones puras de filtrado y orden.	catálogo
`funciones-carrito.js`	Carrito en `localStorage`, totales, envíos, cuotas.	carrito, checkout, agregar
`agregar.js`	Botones "Agregar al carrito" (delegación de eventos) y contador del header.	catálogo, inicio, producto, contacto
`render-blog.js`	Dibuja tarjetas y detalle de artículos del blog.	blog, artículo
`catalogo.js`, `producto.js`, `inicio.js`, `contacto.js`, `carrito.js`, `checkout.js`, `blog.js`, `articulo.js`	Puntos de entrada de cada página.	una página cada uno

```
productos.json → datos.js → (render.js, filtros.js) → catalogo.js / inicio.js / producto.js
                            localStorage ⇄ funciones-carrito.js → carrito.js / checkout.js /
agregar.js
```

La estructura de datos del catálogo es el archivo `assets/data/productos.json`. Para este avance se le agregaron los campos `descripcion` e `ingredientes` a cada producto, que usa la ficha.

4. Eventos JavaScript

Evento	Elemento	Efecto
`change`	Checkboxes de filtros, select de orden	Re-renderiza el catálogo
`submit`	Buscador	Filtra por nombre sin recargar
`reset`	Limpiar filtros	Restaura el catálogo completo
`click` (delegado)	"Agregar al carrito"	Suma al carrito, actualiza el contador, muestra "¡Agregado!"
`click`	Miniaturas de la ficha	Cambia la imagen principal
`click` / `change`	Cantidad (− / +)	Ajusta la cantidad con tope en el stock
`submit`	Calculadora de envío (ficha)	Valida el CP y muestra opciones
`submit`	Formularios de Datos y Entrega	Valida, guarda y avanza de paso
`click`	"Volver" del checkout	Retrocede de paso
`change`	Opciones de envío / medios de pago	Recalcula el resumen y las cuotas
`click`	Fila de cuotas, "Confirmar compra"	Elige plan; genera la confirmación
`click` / `input`	Chips y buscador del blog	Filtra los artículos por categoría y título
`submit` / `input`	Formulario de contacto	Valida; limpia el error al corregir el campo

5. Cambios respecto de la propuesta

- **Checkout en una sola página:** la propuesta planteaba un proceso por pasos; en lugar de un HTML por paso, se resolvió con paneles dentro de `checkout.html` controlados por JS. Se evita recargar la página y se conservan los datos entre pasos.
- **Sin carpeta `pages/`:** el proyecto base del TP4 dejó todos los HTML en la raíz y un único `main.css` (en el SP3 había `pages/` y CSS dividido).
- **Ficha de producto genérica:** pasó de un producto fijo a una plantilla alimentada por `?id=`. Se quitaron el precio anterior tachado y la tabla nutricional, porque el catálogo no tiene esos datos.
- **Persistencia:** al no haber backend, el carrito y el tipo de envío se guardan en `localStorage`. El pedido confirmado no se envía a ningún servidor: la confirmación es simulada y el número de pedido es aleatorio.
- **Blog:** no estaba detallado en la propuesta; se agregó como sección simple con contenido de ejemplo (sin backend ni panel de edición).
- **Contacto:** el formulario valida y confirma, pero no envía el mensaje (requiere backend, fuera del alcance de esta entrega).
- **Regla de visibilidad:** se agregó `[hidden] { display: none !important; }` al CSS para que el atributo `hidden` funcione sobre componentes con `display: grid`.

- **Limitaciones conocidas:** el stock máximo por producto se calcula a partir del texto de stock ("Últimas N unidades"; 10 en el resto), y el carrito parte de un ejemplo precargado la primera vez que se abre el sitio (decisión del módulo de carrito).

6. Verificación

Se probaron los flujos en Chrome con un script automatizado (Playwright): catálogo con búsqueda por URL; blog (navegación, filtros, buscador, artículo y productos relacionados); agregar al carrito y contador; ficha con id válido, inexistente y sin stock; validaciones y mensajes de contacto; recorrido completo del checkout (errores por paso, retiro/envío, volver atrás, transferencia, confirmación y carrito vaciado); ausencia de scroll horizontal en 390 px y de errores de consola.

Anexo — Uso de Inteligencia Artificial

Se utilizó Claude Code (Anthropic) como asistente para analizar el código existente, implementar las vistas y módulos descriptos y generar este informe.